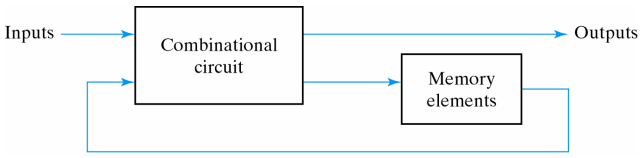
Latches and Flip-Flops

Another class of logic circuits are *sequential circuits*. These circuits are *two-valued* networks in which the outputs at any instant are dependent not only upon the inputs present at that instant but also upon the past history (sequence) of inputs.

Sequential circuits are classified into:

- **Synchronous sequential circuits** – Their behaviour is determined by the values of the signals at only discrete instants of time.
- **Asynchronous sequential circuits** – Their behaviour is immediately affected by the input signal changes.

The block diagram of a sequential circuit is shown below:



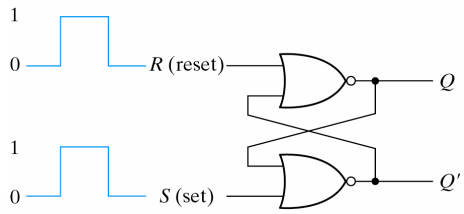
The basic logic element that provides memory in many sequential circuits is the *flip-flop*.

1. Latches

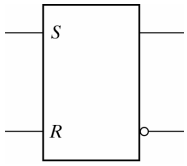
Latches form one class of flip-flops. This class is characterized by the fact that the timing of the output changes is not controlled. Although latches are useful for storing binary information and for the design of asynchronous sequential circuits, they are not practical for use in synchronous sequential circuits.

1.2 The *SR* Latch

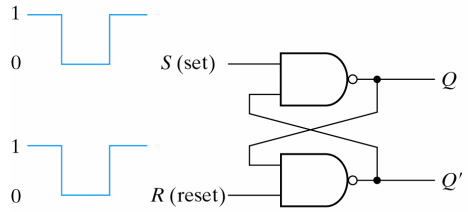
It is a circuit with two cross-coupled NOR gates *or* two cross-coupled NAND gates. The one with NOR gates is shown below:



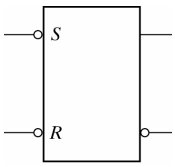
S	R	Q	Q'	
1	0	1	0	
0	0	1	0	(after $S = 1, R = 0$)
0	1	0	1	
0	0	0	1	(after $S = 0, R = 1$)
1	1	0	0	



The *SR* latch with two cross-coupled NAND gated is shown below:



S	R	Q	Q'	
1	0	0	1	
1	1	0	1	(after $S = 1, R = 0$)
0	1	1	0	
1	1	1	0	(after $S = 0, R = 1$)
0	0	1	1	

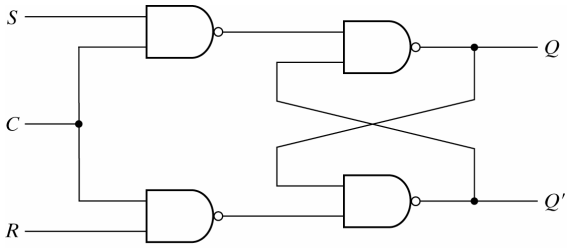


The condition that is undefined is when both inputs are equal to 0 at the same time.

Comparing the NAND with the NOR latch note that the input signals for the NAND require the complement of those values used for the NOR latch.

Because the NAND latch requires a 0 signal to change its state, it is sometimes referred to as an *S'-R'* latch.

The operation of the *SR* latch can be modified to provide an additional control input that determines when the state of the latch can be changed. An *SR* latch with control input is shown below:



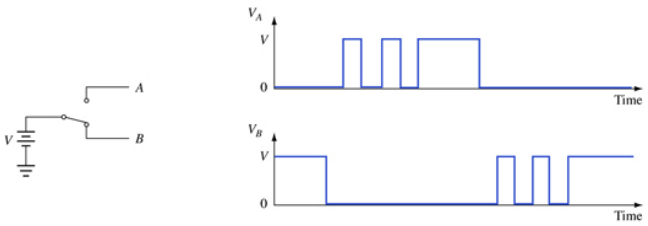
C	S	R	Next state of Q
0	X	X	No change
1	0	0	No change
1	0	1	$Q = 0$; Reset state
1	1	0	$Q = 1$; set state
1	1	1	Indeterminate

The control input *C* acts as an enable signal for the other two inputs.

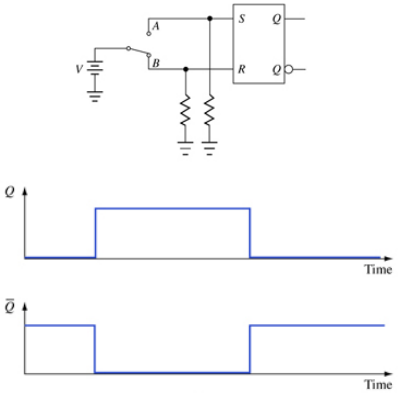
An indeterminate condition occurs when all three inputs are equal to 1. This condition makes the circuit difficult to manage and is seldom used in practice. Nevertheless, it is an important circuit because other latches and flip-flops are realized from it.

1.2 An Application of the *SR* Latch

A common problem involving switches is the occurrence of *contact bounce* (caused by springiness of the contacts):

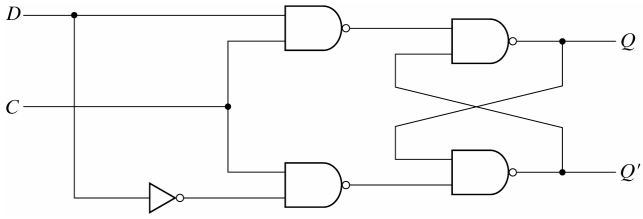


A *switch debouncer* circuit and the corresponding waveforms are shown below:

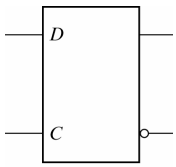


1.3 The Gated *D* Latch

One way to eliminate the undesirable condition of the indeterminate state in the *SR* latch is to ensure that inputs *S* and *R* are never equal to 1 at the same time. This is done by the *D* latch:



C	D	Next state of Q
0	X	No change
1	0	Q = 0; Reset state
1	1	Q = 1; Set state



The *D* input is sampled when *C* = 1. Thus, the *D* latch has the ability to hold data in its internal storage. The output follows changes in the data input as long as the control input is gated.

The circuit is often called a *transparent latch*.

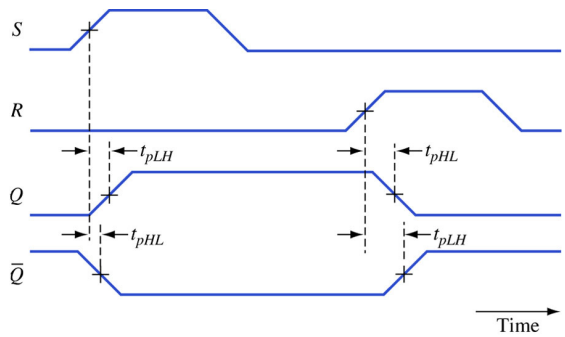
2. Timing Considerations

A convenient way of showing the terminal behaviour of a latch (or a flip-flop) is the timing diagram. This is a graph that depicts the input and output transitions of a latch as a function of time.

2.2 Propagation Delays

The *propagation delay* is the time it takes a change in the input signal to produce a change in output signal.

Propagation delays (t_{pLH} & t_{pHL}) in an *SR* latch are illustrated below:

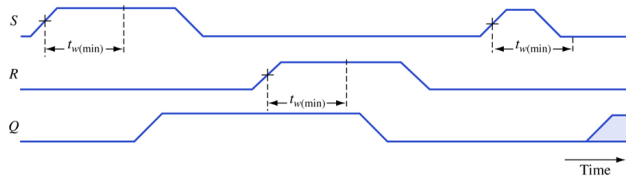


t_{pLH} – from low to high (1 to 0).

t_{pHL} – from high to low (1 to 0).

2.3 Minimum Pulse Width, $t_{w(min)}$

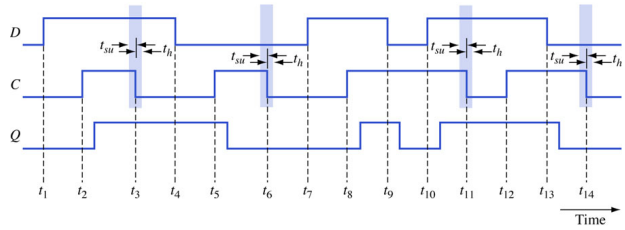
This is the minimum amount of time a signal must be applied in order to produce a desired result:



The shaded area of the output signal *Q* indicates that the state of the latch is unpredictable since the set signal *S* did not satisfy the $t_{w(min)}$ constraint.

2.4 Setup (t_{su}) and Hold Times (t_h)

A timing diagram for a gated *D* latch is:



t_{su} – minimum time the *D* signal must be held fixed before the latching action.

t_h – minimum hold time.

3. Flip-Flops

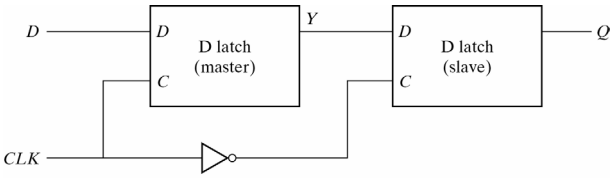
When latches are used for the memory elements in sequential circuits, a serious difficulty arises. Recall that latches have the property of immediate output responses (i.e., transparency).

Because of this the output of a latch cannot be applied directly (or through logic) to the input of the same or another latch when all the latches are triggered by a common clock source.

Flip-flops are used to overcome this difficulty.

3.1 Edge-Triggered *D* Flip-Flop

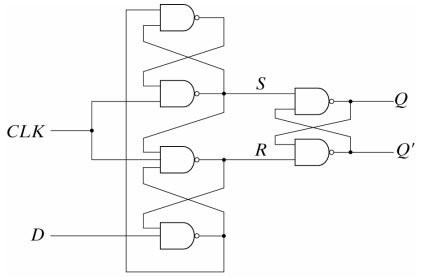
A *D* flip-flop may be realized with two *D* latches connected in a *master-slave* configuration:



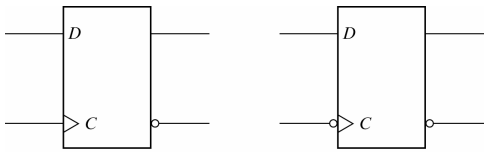
The circuit samples the *D* input and changes its *Q* output *only* at the negative-edge of the controlling clock signal (*CLK*).

It is also possible to design the circuit so that the flip-flop output changes on the positive edge of the clock (transition from 0 to 1). This happens in a flip-flop that has an additional inverter between the *CLK* terminal and the junction between the other inverter and input *C* on the master latch.

An efficient realization of a positive edge-triggered *D* flip-flop uses three *SR* latches:



The graphic symbol for the edge-triggered *D* flip-flop is:

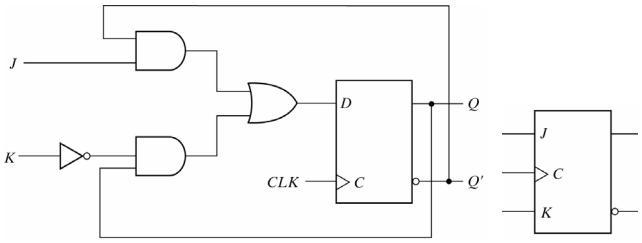


positive-edge negative-edge

3.2 Other Flip-Flops

The most economical and efficient flip-flop in terms of transistor count and silicon area is the *D* flip-flop. Other types of flip-flops can be realized by using the *D* flip-flop and external logic. Two flip-flops widely used in the design of digital systems are the *JK* and the *T* flip-flops.

There are three operations that can be performed with a flip-flop: set it to 1, reset it to 0, complement its output. The *JK* flip-flop performs all three:

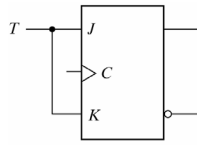


The *J* input sets it to 1, the *K* input resets it to 0, and when both inputs are enabled, the output is complemented.

This can be verified by investigating the circuit applied to the *D* input:

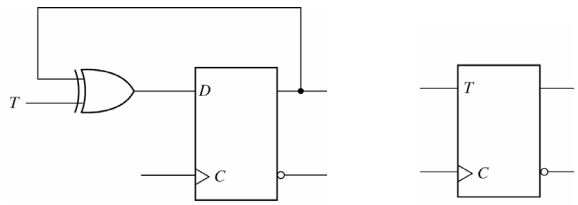
$$\boxed{D = JQ' + K'Q}$$

The *T* (toggle) flip-flop is a complementing flip-flop and can be obtained from a *JK* flip-flop when inputs *J* and *K* are tied together:



When $T = 0$ ($J = K = 0$) a clock edge does not change the output. When $T = 1$ ($J = K = 1$) a clock edge complements the output. The complementing flip-flop is useful for designing binary counters.

The T flip-flop can be constructed with a *D* flip-flop an X-OR gate:



The expression for the *D* input is:

$$\boxed{D = T \oplus Q = TQ' + T'Q}$$

When $T = 0$, then $D = Q$, and when $T = 1$, then $D = Q'$.

3.3 Characteristic Tables

A characteristic table defines the logical properties of a flip-flop by describing its operation in tabular form. The flip-flops characteristic tables are:

JK Flip-Flop

J	K	$Q(t+1)$	
0	0	$Q(t)$	No change
0	1	0	Reset
1	0	1	Set
1	1	$Q'(t)$	Complement

D Flip-Flop

D	$Q(t+1)$	
0	0	Reset
1	1	Set

T Flip-Flop

T	$Q(t+1)$	
0	$Q(t)$	No change
1	$Q'(t)$	Complement

$Q(t)$ refers to the present state prior to the application of a clock edge.

$Q(t + 1)$ is the next state one clock period later.

The clock edge input is not included in the characteristic tables, but is implied to occur between time t and $t + 1$.

3.4 Characteristic Equations

The logical properties of a flip-flop as described in its characteristic table can be expressed also algebraically with a characteristic equation.

For the *D* flip-flop the characteristic equation is:

$$Q(t+1) = D$$

It states that the next state of the output will be equal to the value of input *D* in the present state.

The characteristic equation for the *JK* flip-flop is:

$$Q(t+1) = JQ' + K'Q$$

where *Q* is the value of the flip-flop output prior to the application of a clock edge.

Finally, the characteristic equation for the *T* flip-flop is:

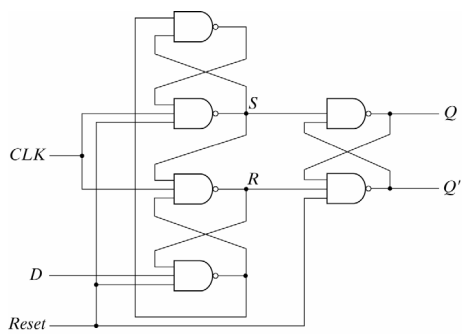
$$Q(t+1) = T \oplus Q = TQ' + T'Q$$

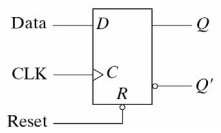
3.5 Direct Inputs

Some flip-flops have asynchronous inputs that are used to force the flip-flop to a particular state independent of the clock. This feature is useful, e.g., when power is turned on in a digital system.

The input that sets the flip-flop to 1 is called *preset* or *direct set*. The input that clears the flip-flop to 0 is called *clear* or *direct reset*.

A *D* flip-flop with asynchronous reset is:





R	C	D	Q	Q'
0	X	X	0	1
1	↑	0	0	1
1	↑	1	1	0